

**SIMATS ENGINEERING**  
**CO3 - ASSESSMENT TOOL - 2: Case Study**

**COURSE NAME:** Cloud Computing and Big Data Analytics      **COURSE CODE:** CSA1522

|                 |                  |
|-----------------|------------------|
| Name            | <b>P. Guru</b>   |
| Register Number | <b>192521010</b> |

**COURSE OUTCOME COVERED**

**CO3:** *Implement cloud-based solutions using cloud storage, web APIs, and platform services for real-world applications. (BL3)*

Dave's Taxonomy: Precision (Level 3)  
Question Count: 3

**Total Marks: 30**

---

**Code of Conduct**

I P. Guru/192521010 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: P. Guru

**Assessment Tasks - Case Study**

**1. Case Study 1: Smart Retail Inventory Management System**

A retail chain implements a cloud-based inventory system connected to store devices. Clients (POS systems and dashboards) communicate via APIs to centralized cloud services. Cloud storage stores product data, transaction logs, and analytics datasets. The system uses platform services for analytics and demand forecasting. Secure access is maintained through VPNs, firewalls, and API gateways. This solution integrates infrastructure, services, and web applications for efficient operations.

**2. Case Study 2: Cloud-Based Document Collaboration System**

An enterprise deploys a document collaboration tool similar to Google Docs. Users access the platform via browsers, enabling real-time editing and sharing. Documents are stored in distributed cloud storage with version control mechanisms. Web APIs enable synchronization across multiple clients and devices. Network infrastructure ensures low latency and high availability globally. Security includes encryption, identity management, and access permissions for collaboration.

**3. Case Study 3: Online Food Ordering Platform**

A startup builds a cloud-based food ordering system accessible via web browsers and mobile clients. Frontend clients interact with backend services through RESTful Web APIs hosted on a cloud platform. Cloud storage is used to store menu images, invoices, and customer data securely. The infrastructure uses virtual machines and managed Kubernetes for scalability and reliability. Security is enforced

through authentication tokens, HTTPS, and role-based access control. The system demonstrates integration of clients, APIs, storage, and platform services in real time.

### Case Study Rubric

| Criteria                | Weightage | Level 4 - Exemplary                                        | Level 3 - Proficient                | Level 2 - Developing     | Level 1 - Beginning           |
|-------------------------|-----------|------------------------------------------------------------|-------------------------------------|--------------------------|-------------------------------|
| Understanding of Case   | 25%       | Clearly identifies all technical and business requirements | Identifies most requirements        | Basic understanding only | Poor understanding of case    |
| Application of Concepts | 25%       | Applies suitable cloud concepts and tools effectively      | Good application with minor issues  | Partial application      | Weak or incorrect application |
| Analysis                | 20%       | Strong analysis with clear trade-off reasoning             | Reasonable analysis                 | Limited analysis         | Minimal analysis              |
| Solution Design         | 20%       | Solution is feasible, practical, and well justified        | Reasonable solution with minor gaps | Basic solution           | Unclear solution              |
| Clarity                 | 10%       | Well-structured and easy to follow                         | Mostly clear                        | Somewhat unclear         | Poorly organized              |

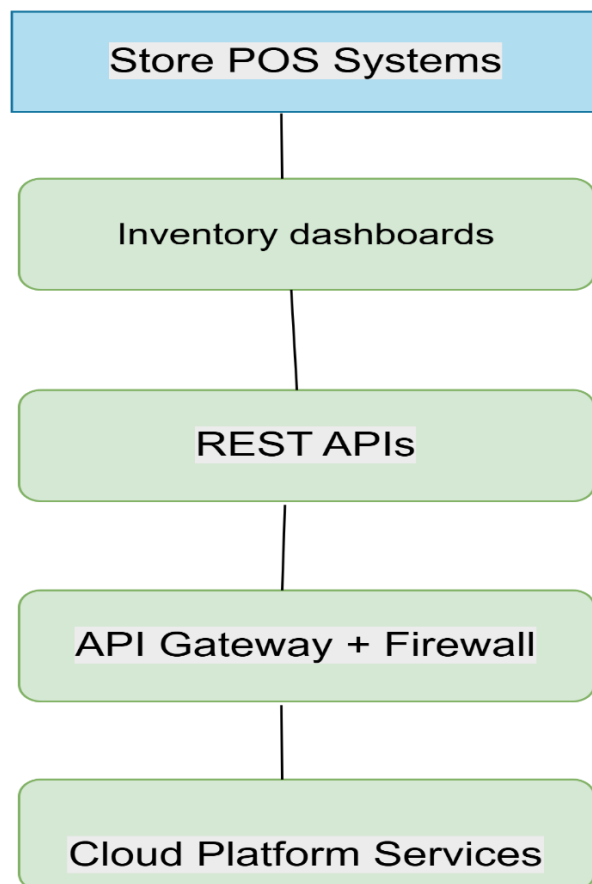
## 1. Case Study 1: Smart Retail Inventory Management System

A retail chain implements a cloud-based inventory system connected to store devices. Clients (POS systems and dashboards) communicate via APIs to centralized cloud services. Cloud storage stores product data, transaction logs, and analytics datasets. The system uses platform services for analytics and demand forecasting. Secure access is maintained through VPNs, firewalls, and API gateways. This solution integrates infrastructure, services, and web applications for efficient operations.

ANS:

- A Smart Retail Inventory Management System is a cloud-based solution that enables retail stores to manage stock efficiently across multiple branches. Instead of maintaining separate inventory systems at each store, all product information, sales transactions, and analytics are stored in a centralized cloud platform. This ensures real-time synchronization, minimizes stock shortages, and improves business decision-making.

**System architecture:**



## **Components:**

### **1. Client Layer:**

- Point of Sale (POS) systems
- Inventory management dashboards
- Mobile applications for managers

Clients send requests to cloud services through secure APIs.

### **2. Web APIs:**

REST APIs provide communication between clients and servers.

Functions include:

- Product lookup
- Inventory updates
- Sales recording
- Report generation

Example:

GET /products

POST /sales

PUT /inventory

DELETE /expired-items

### **3. Cloud Storage:**

Cloud storage stores:

- Product information
- Inventory details
- Sales transactions
- Analytics datasets
- Customer purchase history

Advantages:

- Scalability
- Data durability
- Automatic backup
- Easy accessibility

#### **4. Platform Services (PaaS):**

The cloud platform provides:

- Demand forecasting
- Machine learning analytics
- Sales prediction
- Automated reporting

Benefits:

- Better inventory planning
- Reduced overstock
- Reduced product wastage

#### **5. Infrastructure (IaaS):**

Infrastructure includes:

- Virtual Machines
- Networking
- Storage Servers
- Load Balancers

Provides computing resources for running applications.

#### **6. Security:**

Security mechanisms include:

- VPN
- Firewall
- API Gateway
- HTTPS
- User Authentication
- Authorization

These prevent unauthorized access.

### **Working Process:**

1. Customer purchases a product.
2. POS updates inventory.
3. Request goes through API Gateway.
4. Cloud service processes request.
5. Cloud storage updates inventory.
6. Analytics engine updates reports.
7. Demand forecasting predicts future stock requirements.
8. Managers monitor dashboards.

### **Advantages:**

- Centralized inventory management
- Real-time stock updates
- Easy scalability
- Reduced operational cost
- Improved forecasting
- Better customer satisfaction

## **Applications**

- Supermarkets
- Shopping malls
- Pharmacy chains
- Electronic stores
- Fashion retail

## **2. Case Study 2: Cloud-Based Document Collaboration System:**

**An enterprise deploys a document collaboration tool similar to Google Docs. Users access the platform via browsers, enabling real-time editing and sharing. Documents are stored in distributed cloud storage with version control mechanisms. Web APIs enable synchronization across multiple clients and devices. Network infrastructure ensures low latency and high availability globally. Security includes encryption, identity management, and access permissions for collaboration.**

**ANS:**

- A Cloud-Based Document Collaboration System allows multiple users to create, edit, and share documents simultaneously over the internet. Similar to Google Docs, users access documents through web browsers while all files are securely stored in distributed cloud storage. Real-time synchronization ensures that every participant sees updates instantly.

### **Components:**

#### **1. Clients:**

Users access documents through:

- Browsers
- Mobile applications
- Desktop applications

#### **2. Web APIs**

**REST APIs provide:**

- Login
- Document creation
- Sharing
- Editing
- Synchronization

Example APIs:

- POST /login
- GET /document
- PUT /document
- POST /share

### **3. Distributed Cloud Storage**

#### **Stores:**

Documents

- Images
- Attachments
- User settings
- Backup copies

Advantages:

- High availability
- Data replication
- Fault tolerance

### **4. Version Control:**

Every edit creates a new version.

#### **Benefits:**

- Restore previous versions
- Track changes
- Prevent accidental data loss
- Collaboration history

### **5. Network Infrastructure:**

#### **Provides:**

- Global servers
- Content Delivery Network (CDN)
- Load balancing
- Low latency

Ensures smooth collaboration.

### **6. Security:**

#### **Includes:**

- Encryption
- Identity Management
- Multi-Factor Authentication
- Access Permissions
- HTTPS

Protects confidential documents.

### **Working Process:**

1. User logs in.



2. Browser sends request to cloud.
3. Document loads from cloud storage.
4. User edits document.
5. Changes are synchronized through APIs.
6. Version control saves every modification.
7. Other users receive updates instantly.

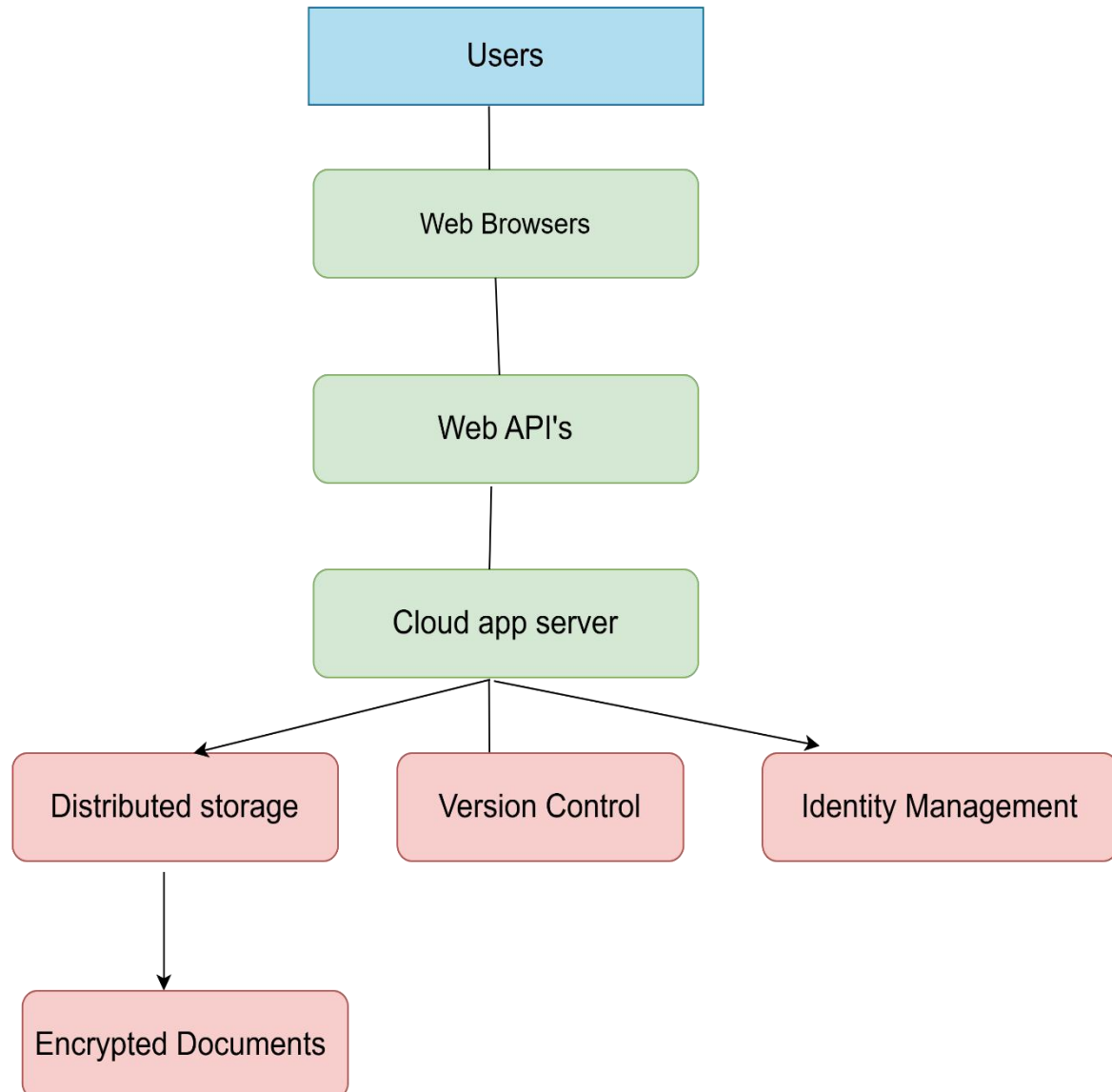
**Advantages:**

- Real-time collaboration
- Automatic backup
- Easy document sharing
- High availability
- Version tracking
- Secure access

**Applications:**

- Educational institutions
- Software companies
- Government offices
- Banking
- Corporate organizations

## System Architecture:



### **3. Case Study 3: Online Food Ordering Platform**

**A startup builds a cloud-based food ordering system accessible via web browsers and mobile clients. Frontend clients interact with backend services through RESTful Web APIs hosted on a cloud platform. Cloud storage is used to store menu images, invoices, and customer data securely. The infrastructure uses virtual machines and managed Kubernetes for scalability and reliability. Security is enforced through authentication tokens, HTTPS, and role-based access control. The system demonstrates integration of clients, APIs, storage, and platform services in real time.**

**ANS:**

- An Online Food Ordering Platform enables customers to browse restaurant menus, place orders, make payments, and track deliveries using web browsers or mobile applications. The application runs on cloud infrastructure to provide scalability, reliability, and high availability during peak demand.

#### **Components:**

##### **1. Client Layer**

Includes:

- Web browser
- Android application
- iOS application

Functions:

- Browse menu
- Place order
- Track delivery
- Online payment

##### **2. RESTful APIs:**

REST APIs perform:

- User registration
- Login
- Order placement
- Payment processing

- Delivery tracking

Example:

POST /login

GET /menu

POST /order

GET /track-order

### **3. Cloud Storage**

Stores:

- Menu images
- Customer information
- Invoices
- Order history
- Restaurant details

Benefits:

- Secure storage
- Automatic backup
- Easy retrieval

### **4. Infrastructure:**

Uses:

- Virtual Machines
- Managed Kubernetes
- Auto Scaling
- Load Balancers

Benefits:

- Handles thousands of users

- High reliability
- Fault tolerance
- Continuous availability

## **5. Platform Services**

Provides:

- Payment integration
- Notification service
- Order analytics
- Customer recommendations
- Monitoring services

## **6. Security**

Security mechanisms include:

- HTTPS
- Authentication Tokens (JWT)
- Role-Based Access Control (RBAC)
- Password Encryption
- Secure API Gateway

Roles:

- Customer
- Restaurant
- Delivery Partner
- Administrator

Each role has specific permissions.

---

## **Working Process**

1. Customer logs into the application.
2. Menu is retrieved through REST APIs.
3. Customer places an order.
4. Cloud platform processes the request.
5. Restaurant receives the order.
6. Invoice and customer details are stored in cloud storage.
7. Delivery partner receives the delivery request.
8. Customer tracks the order in real time.
9. Order status is continuously updated through APIs.

**Advantages:**

- Easy online ordering
- Scalable cloud infrastructure
- High availability
- Faster order processing
- Secure payment transactions
- Real-time order tracking
- Improved customer experience

**Applications:**

- Food delivery startups
- Restaurant chains
- Grocery delivery services
- Cloud kitchens
- Catering businesses

## System Architecture:

